

重庆理工大学毕业设计（论文）

文 献 翻 译

毕业设计
论文题目

网络流量检测管理系统的设计与开发

原文标题

Model of Load Distribution Between Web Proxy Servers Using Network Traffic Analysis

译文标题

基于网络流量分析的 Web 代理服务器负载分配模型

原文出处

SN Computer Science, 2020, 1:103. DOI:10.1007/s42979-020-0108-7

学 院	计算机科学与工程学院		
专 业	计算机科学与技术	班 级	122030701

学生姓名	郭子谊	学 号	11912060207
指导教师	范伟		

译文要求

- 1、译文内容必须与课题（或专业）内容相关。
- 2、译文不少于 2000 字；外文阅读量至少 3 篇（10 万外文字符以上）。
- 3、参考文献不翻译，不打印。
- 4、译文原文（或复印件）建议一张 A4 打印 2 页内容，并双面打印，附在译文后备查。

译文评阅

评价维度	评价结果		
内容准确性	☐ 忠实原文，无遗漏或错误	☐ 存在少量内容偏差	☐ 内容翻译错误较多
语言流畅性	☐ 译文通顺，符合中文习惯	☐ 较通顺，部分生硬表达	☐ 不通顺，影响理解
专业术语翻译	☐ 术语准确，符合学科规范	☐ 部分术语翻译不规范	☐ 术语翻译错误或混乱
逻辑连贯性	☐ 逻辑清晰，结构合理	☐ 逻辑较清晰，部分衔接欠佳	☐ 逻辑混乱，层次不清
综合评价等级	☐ 优秀	☐ 良好	☐ 合格 ☐ 不合格
其他建议			

指导教师签字： (手签名) 年 月 日

基于网络流量分析的 Web 代理服务器负载分配模型

B. S. Renuka, G. T. Prafulla Shashikiran

发表时间：2020 年 3 月 28 日

摘要

代理服务器用于分别向提出请求的客户端节点或终端提供信息访问服务。在当前框架下，当大量客户端同时访问 Internet 时，代理服务器由于自身处理能力有限，容易遇到性能瓶颈。由于互联网流量具有突发性，Web 服务器也经常面临过载情况。本文的重点是为 Web 服务器提出有效的负载控制机制。在负载控制中，一个重要问题是尽量减少代理服务器在分配负载并转发到主服务器时所消耗的工作量。本文通过研究服务器与代理服务器之间网络链路上的负载，提出一种用于代理服务器负载均衡的方法。该研究基于 HTTP/HTTPS、TCP 等端口相关的数据包与套接字每秒比特数移动量进行计算。所提出的方法能够通过降低代理服务器负载来提升系统性能。

关键词：负载均衡；代理服务器；HTTP Web 协议

1 引言

负载均衡是将整体负载分配到服务器系统中所有请求节点或客户端的过程，当前相关技术的基本使用方式如图 1 所示。在所有渲染过程和网络问题中，都有必要了解资源利用率和响应时间。云环境中的负载分配依赖于负载均衡算法。系统必须考虑网络通道中的网络流量、数据传输以及资源分配，这也被称为负载分配技术。客户端连接到代理服务器并请求某种服务，例如文件、模块、网页或其他可由另一台服务器提供的资源；代理服务器则对该请求进行评估，以便简化并控制其不可预测性^[1]。

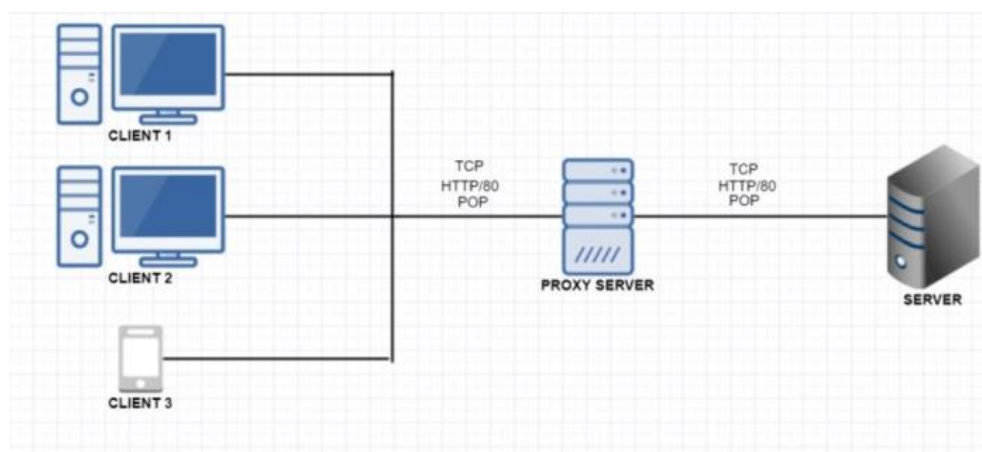


图 1 反向代理服务器表示模型

最优负载分配是启动任何负载分配算法的核心目标，通常包括最大吞吐量、低响应时间和低开销三个主要指标。这三个指标是开发负载分配算法时的主要标准。代理服务器上的负载均衡器在服务器与客户端之间充当接口。负载均衡主要用于简化访问过程：当客户

端请求某个网页时，系统通过某种算法将请求路由到某台服务器，因此它相当于客户端与服务器之间的单一接触点。通过 TCP 多路复用实现的抽象、故障切换、响应能力、错误报告、无缝恢复、可扩展性和可复用性，是该技术的主要优势。

一般而言，基于自治代理的自组织代理架构，可以类比为一个简单的市场买卖环境。真实买方相当于客户端，它会为自己的每一次请求选择相同商店，类似于浏览器中预先定义的代理；而市场扩展完全依赖卖方，即自治代理。每个商店都具有类似缓存的有限本地库存，其目标是扩大客户数量。良好的服务可以通过两种方式提供：一种是本地库存中已经拥有所请求的对象，另一种是知道供应该对象的最合适途径。此外，每个代理都会通过专注于某一类对象，也就是聚类，来吸引更多请求或其他代理。这一选择通常取决于商店当前的专业化方向和传入请求模式。上述动态市场中的买卖关系，与分布式自治代理系统中提高代理请求命中率的目标具有较强相似性。因此，前提是需要有一种有效方法来观察并分类传入请求。就哈希算法而言，简单的模运算可以在请求 URL 上定义相似性；但是，这种简单安排仍然不够充分^[2]。

由协作代理系统在本地完成请求解析时，平均命中率越高，说明系统越能够把最需要的 Web 对象迁移到更接近客户端的位置。平均跳数表示解析一个请求所需要经过的跳转次数；平均跳数越低，请求解析耗时越短。在客户端与代理、代理与代理、代理与服务器之间转发请求的动作，均构成一次跳转。

因此，本文的目标是通过分析代理服务器周边网络通道的网络流量，减少负载分配算法的运行时间，从而降低代理服务器负载。本文采用反向代理服务器方法，即传入请求由中间代理处理，代理代表客户与服务器上的服务进行交互。反向代理最常见的用途，是为 Web 应用和 API 提供负载均衡。Web 代理服务器服务中的“每秒请求数”计数器，表示代理服务器每秒代表客户端评估的请求数量。该计数器体现了代理服务器正在承担的工作量，其数值会根据客户端请求类型的不同而显著变化。Web Security and Acceleration Server 在代理服务器方面也具有多种基础功能^[3]。

2 负载均衡算法中使用的技术

在随机分配、最少计数等多种算法技术中，轮询算法是许多行业经常采用的方法。轮询算法的逻辑是：对于 k 台服务器和 i 个请求，按照 $i \bmod k$ 的顺序工作；因为对于所有 $k+1$ 个请求，请求又会回到第一台服务器。采用该方法时需要重点考虑的改进问题，是如何处理不同延迟，因为延迟差异会实际影响系统的响应退化。为解决该问题，有研究利用对数正态分布降低延迟，同时指出在推导对数正态分布时，延迟不能为零^[4]。

TCP 套接字上的 HTTP/HTTPS 请求是代理服务器与主服务器之间常见的通信方式。因此，这类通信通常通过 TCP 套接字承载 HTTP/HTTPS 请求完成。通过研究 HTTP 请求头中内容大小，并分析每秒比特数 (bps) 的数据量，就可以使用并评估该通信链路。本文提出的方法扩展了这一部分，并将其作为主要依据。

3 相关工作

WebSocket 握手在会话建立阶段利用了 HTTP 协议^[5]。WebSocket 升级请求是一个常规 HTTP GET 请求，会把会话端点指定为请求 URI 的组成部分，如图 2 所示^[6]。

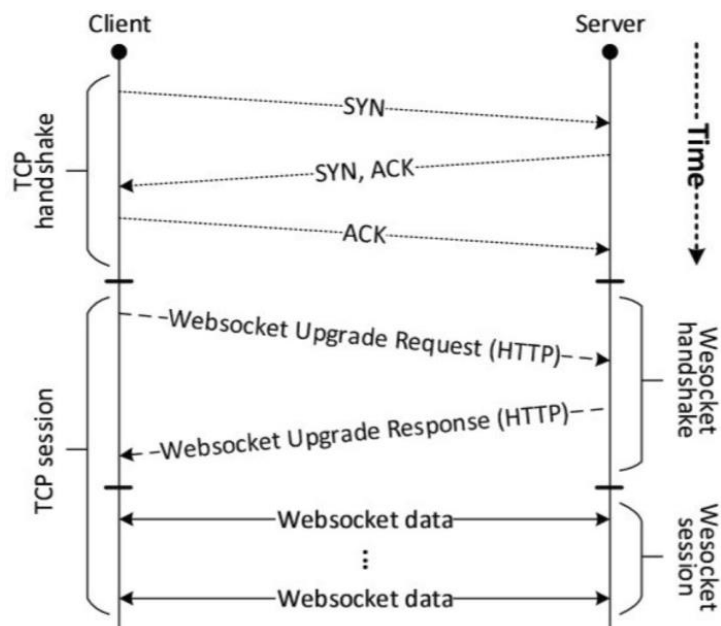


图 2 TCP 上的 WebSocket 时序图

在长时间运行的会话中，初始握手带来的影响会减弱，唯一可以观察到的流量开销来自 WebSocket 数据帧。当给定数量的载荷数据使用普通 TCP 协议传输时，这些数据会直接嵌入 TCP 载荷中。然而，当数据以 WebSocket 帧传输时，TCP 载荷同时包含实际载荷数据和 WebSocket 帧头^[5]。每台代理服务器上的工作负载可以根据日志记录数量估算。为了度量本文工作的性能，相关研究在真实环境中进行了一组实验，并使用从大学代理日志获得的真实流量模拟数据集^[7]。此外，多个行业中还使用了许多负载均衡技术，例如故障转移负载均衡、最优负载均衡、比例负载均衡、饱和负载均衡和搜索过滤负载均衡^[8]。

当普通 HTTP 请求通过 HTTP 代理发起时，后续多个阶段都会受到影响。图 3 展示了客户端对 `www.abc.com` 发起代理请求时的通信示例通道。连接时间是指连接到代理服务器所需的时间；下一步是客户端向代理发送 HTTP 请求。例如，DNS 时间用于解析代理主机名，而不是解析 `www.abc.com`；随后代理需要将 `www.abc.com` 解析为 IP 地址，打开到该 IP 地址的连接，并把客户端 HTTP 请求转发给 Web 服务器。代理随后在接收到数据时把信息转发回客户端。通常，等待时间增加意味着 Web 服务器没有足够资源快速服务请求。遗憾的是，在使用代理时，很难判断此类问题究竟来自 Web 服务器资源不足、较慢的 DNS 服务提供商，还是有损网络导致连接时间增加^[3]。从客户端角度看，原始 DNS 时间和连接时间已经被合并进等待时间，因此不再可能独立测量这些指标。

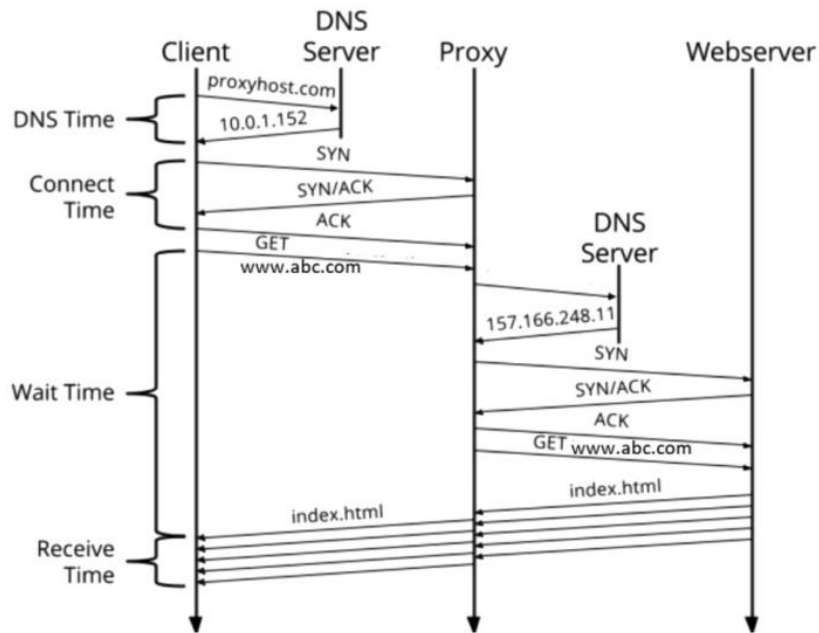


图 3 代理 HTTP 请求分解

分布式方法依赖于类似 Cache Array Routing Protocol (CARP) 的哈希计算。请求页面通过哈希框架映射到某个代理中，该页面要么由本地缓存解析，要么向源服务器请求。基于哈希的分区通常被视为存储网页的理想方法，因为网页位置是预先定义的；但其主要缺点在于可索引性和适应性较差^[9]。在异构计算框架中，系统在 I/O、CPU、内存等不同负载下的性能，依赖 IOCM 动态负载均衡计算。集群框架中存在多种独特的负载均衡策略，其效率取决于连接各节点的通信拓扑。相关结果表明，该方法可有效处理 I/O 密集型、CPU 密集型和内存密集型任务^[10]。在 Linux 平台上运行的 Squid 代理，在操作系统环境方面比 Windows 平台表现更好。通过 Mozilla Firefox 访问 Linux 服务器时，其平均响应时间分别为 21.9、29.9、20.9、48.2、28.3 秒；使用 Internet Explorer 时分别为 28.9、25.3、25.4、32.9、24.5 秒^[11]。

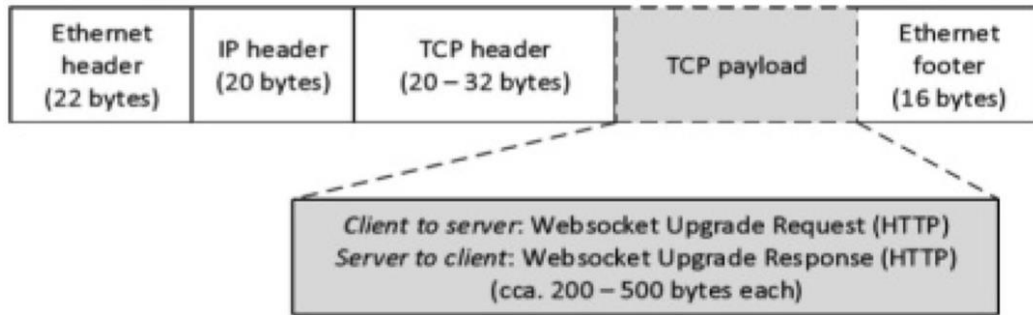
现有协作代理系统可以组织为层次式和分布式两类。层次式方法基于 Internet Caching Protocol (ICP) 和固定层次结构。当某个代理服务器的本地缓存中没有目标页面时，会首先向同一层级的相邻代理请求。如果请求无法在本地解析，则查询层次结构中的根代理，并持续向上查询，直到找到请求对象。这种方式通常会在主根服务器处形成瓶颈^[12]。TCP 客户端只需确保套接字能够打开；也可以配置 HTTP 客户端向后端服务提交有效 HTTP 请求，并定义 HTTP GET、PUT、POST 或 DELETE 操作。HTTP 监控调用的响应必须与配置设置相匹配。

4 提出的方法

本文方法的核心，是降低代理服务器上的负载。这里的负载，是指轮询算法以及其他始终运行在代理服务器上的任务调度算法所造成的实际算法开销。

如图 4 所示，多个节点连接到多个代理服务器。每个客户端请求的信息各不相同；

如果所请求的信息不在代理服务器缓存中，则代理服务器与核心服务器之间的通信会发挥作用。本文提出的方法正是针对这一部分。



a) Network packet structure during the *WebSocket* handshake

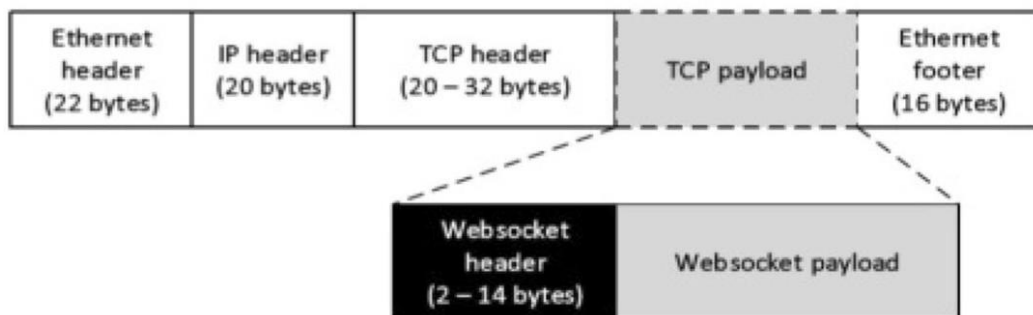


图 4 WebSocket 会话中的网络数据包结构

根据前文讨论的 m 个球和 n 个桶问题，并将其应用到本文场景中，可以得到 m 次命中分布在 n 台服务器上的概率。设有 m 个客户端和 n 台服务器。考虑以下推导，令 Y 表示服务器数量， X 表示客户端数量。

若服务器处于低负载服务状态，则 $X_i = 1$ ；否则 $X_i = 0$ ， $\forall i$ 。

客户端请求命中代理服务器的情况是未知的。对两种可能条件分别取 $E(x)$ ，可以得到如下形式。

$$E(x) = E[\sum X_i] = n(1 - 1/n)^m$$

$$\text{若 } n = m, \text{ 则 } E[X] = n(1 - 1/n)^n$$

上述公式来自对式 (1) 中两种情况分别取 X 的概率函数。同时，还需要关注 $(1 - 1/n)$ 的取值，使其不会随着 m 的增加而不合理地下降。当负载分配算法为随机化算法，且客户端命中代理服务器或代理服务器命中核心服务器时，该公式适用于此类情况。

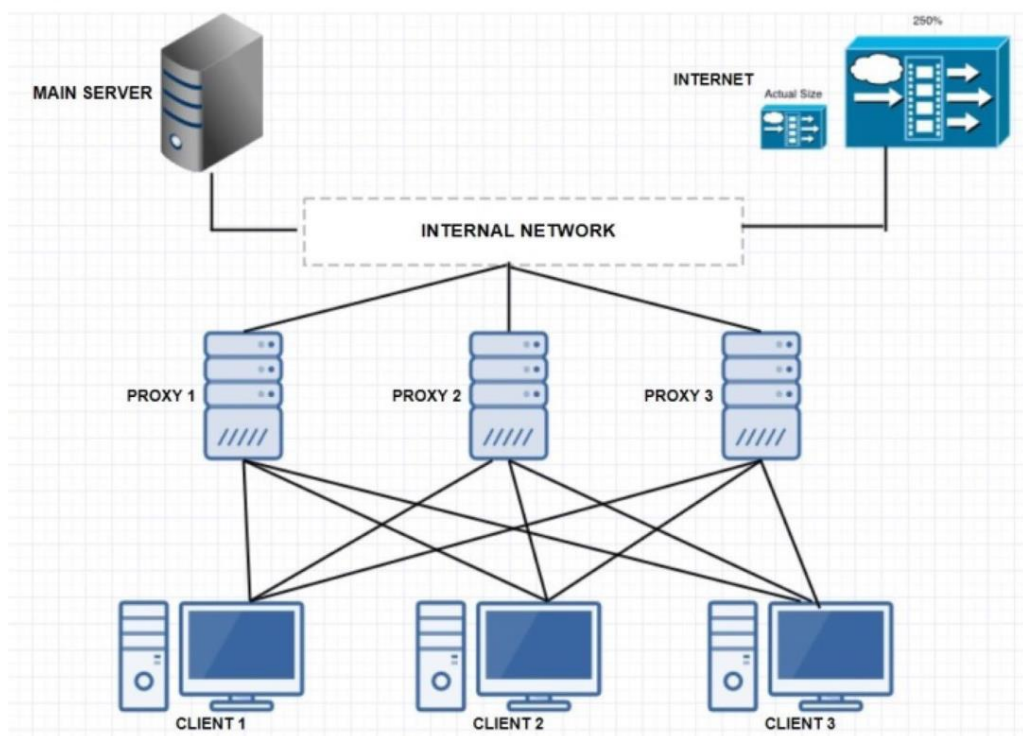


图 5 各节点间工作负载分配流程

4.1 代理服务器与主服务器之间的通信

本小节讨论主服务器与代理服务器之间的网络通信。当请求到达代理服务器后，如果请求的数据，例如来自节点 1 的数据，不能由代理服务器自身解析，则该请求会通过 TCP/IP HTTP 套接字转发到主服务器。在内部计算中，通过检查 HTTP 头部的载荷长度，可以得到经由 TCP/IP 传输的套接字准确大小。通过测量主服务器与特定代理服务器之间通信网络的每秒比特数 (bps)，可以检测代理服务器的负载，以决定是否在代理服务器上运行算法。降低代理服务器负载的主要思路，是仅在需要时才初始化并运行代理服务器上的算法，也就是仅当过多节点连接到特定代理服务器时才运行。图 6 对此进行了说明^[1]。因此，实施所提出过程的步骤如下。

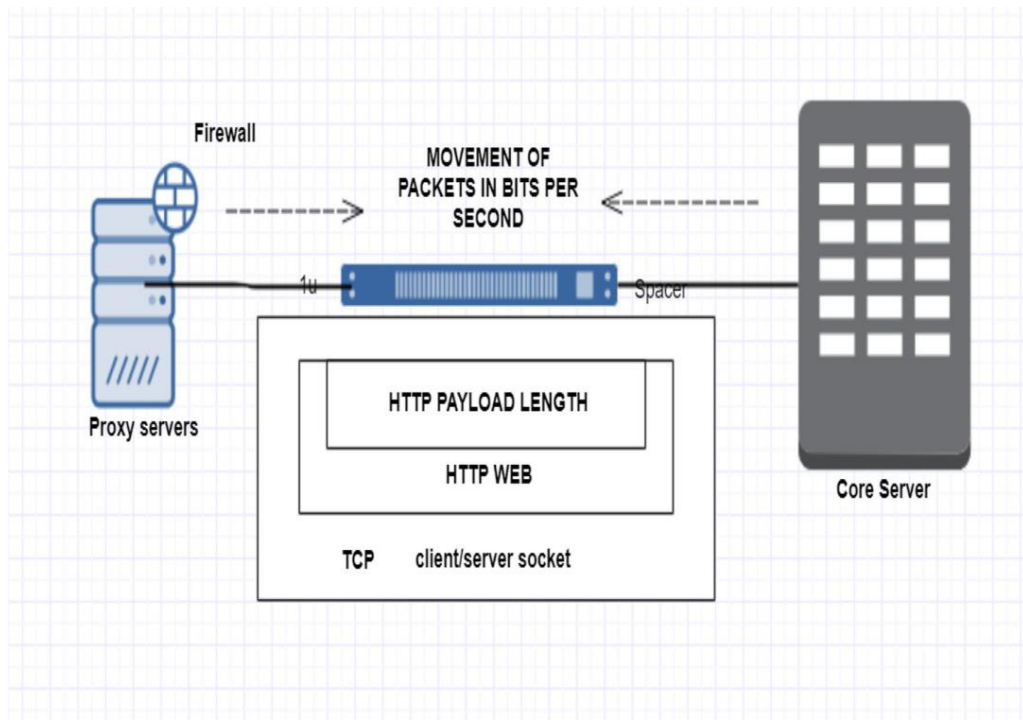


图 6 网络通道实体

步骤 1：当客户端请求信息或原始 Web 数据时，请求会发送到代理服务器。代理服务首先以缓存数据的形式检查自身是否存在所请求的数据或信息。这里存在两种情况：

（a）如果代理服务器中存在请求数据，则通过 HTTP Web 协议以状态码 200 将信息返回给客户端；（b）如果代理服务器中不存在请求数据，则需要执行后续任务。

步骤 2：如果上一步属于情况（b），则研究代理服务器与主服务器之间每条通信网络上的 HTTP 载荷长度大小。载荷长度与网络连通性的乘积以 bps 为单位，用于得到网络强度值。因此，用于计算代理与主服务器之间高速分组接入（HSPA）的表达式为：

$$H(t) = \text{HTTP 端口载荷长度} \times \text{HSPA (bps)}$$

步骤 3：每台代理服务器都显式或隐式运行一个负载均衡器，其中包含轮询、最少计数等任务调度算法。根据前一步结果检查通道中的流量是否超过给定阈值。如果超过阈值，则开始在代理服务器上运行负载分配算法，从而最终降低负载并提高代理服务器性能。为进一步说明上述公式，实验捕获了约 20 分钟场景下的数据包，如图 7 所示^[7]。与其他数据包长度相比，高接近度的数据包长度在整体场景中约占 40%。根据式（3），结合总体平均计数可得 HSPA 为 8200。H(t)=32.8%，这意味着该特定服务器通道吸收了 32% 的带宽，因此约 32% 的较低生成键会被发送到该服务器。

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst start
Packet Lengths	166756	707.53	42	1494	0.0222	100%	1.6900	647.655
0-19	0	-	-	-	0.0000	0.00%	-	-
20-39	0	-	-	-	0.0000	0.00%	-	-
40-79	5498	50.06	42	78	0.0007	3.30%	0.5000	6574.748
80-159	64391	136.59	81	157	0.0086	38.61%	0.6000	7039.020
160-319	19893	207.28	160	319	0.0026	11.93%	1.1300	647.685
320-639	5722	432.91	320	635	0.0008	3.43%	0.1200	7021.983
640-1279	3315	914.28	640	1275	0.0004	1.99%	0.1200	7025.281
1280-2559	67937	1461.41	1280	1494	0.0090	40.74%	0.8200	1113.546
2560-5119	0	-	-	-	0.0000	0.00%	-	-
5120 and greater	0	-	-	-	0.0000	0.00%	-	-

图 7 网络通道实体统计结果

5 分析与讨论

数据包长度会影响网络通道中的延迟,因为它给出了 HTTP 头部中的载荷长度。因此,被称为突发速率的最大数据传输率,会随着最大限制而逐渐增加,如图 6 所示。

服务器与其他服务器之间接触点数量增加,会由于与每台服务器及其网络之间三次握手数量的增加而提高 tcp.errorlog。系统还需要处理生存时间 (TTL),避免连接超出其最大范围。图 8 显示,当通过其他服务器路由的可达性和负载增加时, tcp.errorlog 随之增加。

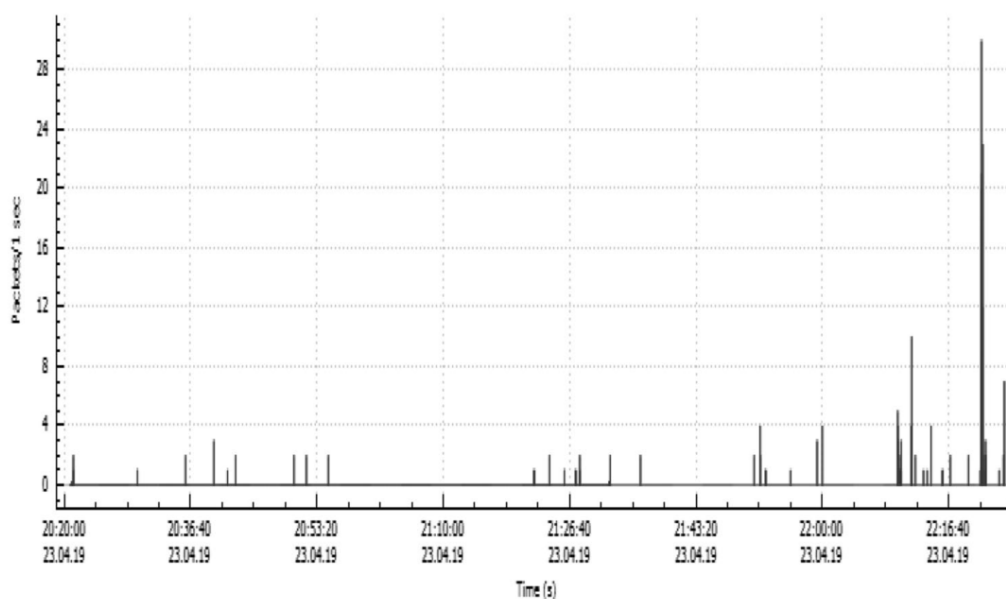


图 8 开销导致 tcp.errorlog 增加

当服务器受到其他代理服务器或核心服务器介入时,网络超时 $H(t)$ 会增加。与 DNS 路由类似,一台服务器会响应其他服务器的数据交换请求。连接路径越长,负载均衡器对特定请求的控制能力越弱,因此该请求会进入不合适的状态。如下图所示,服务器会达到其内部协作关系。

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate
▼ HTTP Responses by Server Address	231				0.0000	100%	0.0200
▼ 192.168.0.1	230				0.0000	99.57%	0.0200
OK	230				0.0000	100.00%	0.0200
▼ 143.166.11.200	1				0.0000	0.43%	0.0100
KO	1				0.0000	100.00%	0.0100
▼ HTTP Requests by Server	2042				0.0003	100%	0.0800
▼ HTTP Requests by Server Address	2042				0.0003	100.00%	0.0800
▼ 239.255.255.250	2040				0.0003	99.90%	0.0800
239.255.255.250:1900	2040				0.0003	100.00%	0.0800
▼ 192.168.0.1	1				0.0000	0.05%	0.0100
192.168.0.1:1900	1				0.0000	100.00%	0.0100
▼ 143.166.11.200	1				0.0000	0.05%	0.0100
lyncdiscover.com	1				0.0000	100.00%	0.0100
▼ HTTP Requests by HTTP Host	2042				0.0003	100.00%	0.0800
▼ lyncdiscover.com	1				0.0000	0.05%	0.0100
143.166.11.200	1				0.0000	100.00%	0.0100
▼ 239.255.255.250:1900	2040				0.0003	99.90%	0.0800
239.255.255.250	2040				0.0003	100.00%	0.0800

图 9 实时配置下服务器间的负载分配

在考虑响应性这一重要方面的同时，还需要考虑物理设备上的负载分配以及 ESX 服务器的安装。实验中，在物理设备上创建虚拟环境，并让 20 个并发用户执行数据检索或上传，以获得最优结果。图 10 给出了平均响应时间。根据获得的结果可以得出结论：对于任何用户而言，任意时刻某一特定线程的响应时间都不是恒定的，而取决于命中次数和吞吐量。当负载增加，或向物理设备注入日志或数据包时，由于 CPU 利用时间、I/O 活动和存储状态变化，响应时间会增加。

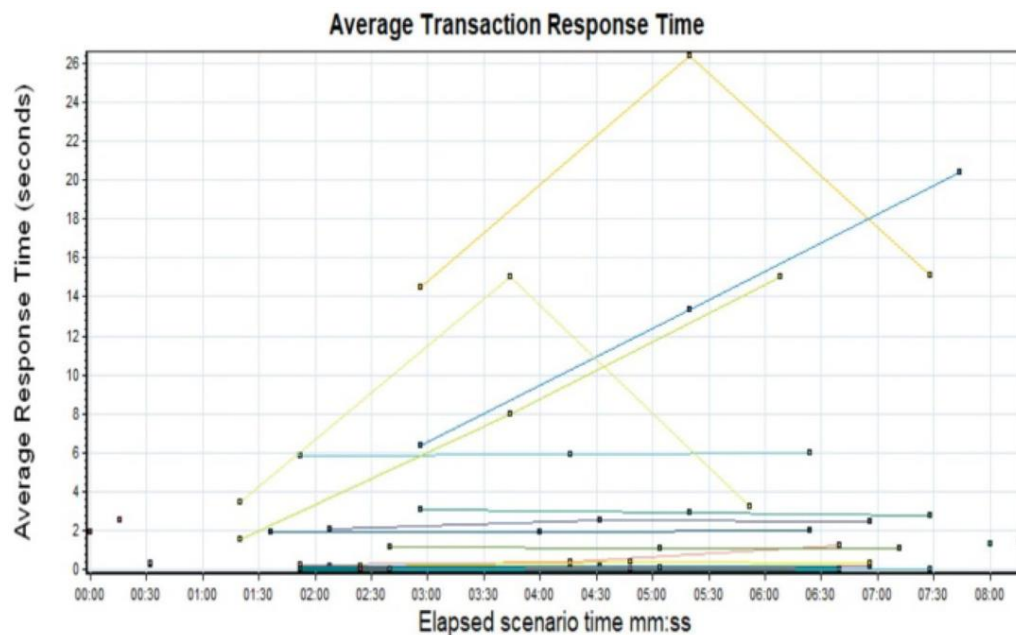


图 10 不同并发用户的平均响应时间

本文通过实验分析，研究如何在不增加额外复杂度的情况下，在网络中引入自组织机

制，并获得更好的响应时间。相关计数器可以测量当前连接到代理服务器 **Web Proxy Service** 的用户数量。该值并不总是意味着用户正在主动使用多个浏览器，用户也可能只是将浏览器置于桌面后台^[13]。

6 结论

当代理服务器承载多个客户端或节点时，在响应时间、吞吐量、资源利用率和开销方面往往会出现较低的性能因子。本文提出的方法提供了一种有效的负载均衡技术，可通过 **Web HTTP** 将请求发送到目标代理服务器，并选择当前在所有代理服务器中工作负载最低的服务器。每台代理服务器的工作负载根据网络通道上的日志记录数量进行估算。通道负载通过 **HTTP** 载荷和数据 **bps** 移动量确定。负载分配算法只有在网络通道流量过高并满足给定阈值时才运行。由于调度器仅在需要时运行，代理服务器不会产生额外开销，负载分配过程更加简便，代理服务器性能也随之提高。